

AIM:Implementing Substitution and Transposition Ciphers

NOTE: Use both the GUI and CLI methods.

1.A. Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques

INPUT: CODE FOR CLI METHOD (Caesar Cipher and Monoalphabetic Cipher)

alphabet = "abcdefghijklmnopqrstuvwxyz"

Caesar Cipher

def caesar_encrypt(message, shift):

result = ""

for letter in message.lower():

if letter in alphabet:

old_position = alphabet.index(letter)

new_position = (old_position + shift) % 26

result += alphabet[new_position]

else:

result += letter

return result

def caesar_decrypt(message, shift):

return caesar_encrypt(message, -shift)

Monoalphabetic (Keyword) Cipher

def build_key_alphabet(keyword):

keyword = keyword.lower()

seen = []

for letter in keyword:

if letter.isalpha() and letter not in seen:

seen.append(letter)

for letter in alphabet:

if letter not in seen:

seen.append(letter)

return "".join(seen)

def mono_encrypt(message, keyword):

key_alphabet = build_key_alphabet(keyword)

result = ""

for letter in message.lower():

if letter in alphabet:

NAME: DEESHA CHAVAN

ROLL NO.: T076

```
    position = alphabet.index(letter)
    result += key_alphabet[position]
else:
    result += letter
return result

def mono_decrypt(message, keyword):
    key_alphabet = build_key_alphabet(keyword)
    result = ""
    for letter in message.lower():
        if letter in key_alphabet:
            position = key_alphabet.index(letter)
            result += alphabet[position]
        else:
            result += letter
    return result

# Simple menu-based CLI
print("=== Classical Cipher Toolkit ===")
print("1. Caesar Cipher - Encrypt")
print("2. Caesar Cipher - Decrypt")
print("3. Monoalphabetic Cipher - Encrypt")
print("4. Monoalphabetic Cipher - Decrypt")

choice = input("Choose 1, 2, 3 or 4: ")
message = input("Enter your message: ")

if choice == "1":
    shift = int(input("Enter shift number (e.g. 3): "))
    print("Encrypted message:", caesar_encrypt(message, shift))
elif choice == "2":
    shift = int(input("Enter shift number (e.g. 3): "))
    print("Decrypted message:", caesar_decrypt(message, shift))
elif choice == "3":
    keyword = input("Enter keyword (e.g. SECURITY): ")
    print("Encrypted message:", mono_encrypt(message, keyword))
elif choice == "4":
    keyword = input("Enter keyword (e.g. SECURITY): ")
    print("Decrypted message:", mono_decrypt(message, keyword))
else:
    print("Invalid choice. Please enter 1, 2, 3 or 4.")
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY

OUTPUT:

```
"IDLE Shell 3.14.6"
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:6c63a6c6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: D:/T076/CYBER PRAC 1.py =====
=== Classical Cipher Toolkit ===
1. Caesar Cipher - Encrypt
2. Caesar Cipher - Decrypt
3. Monoalphabetic Cipher - Encrypt
4. Monoalphabetic Cipher - Decrypt
Choose 1, 2, 3 or 4: 1
Enter your message: home
Enter shift number (e.g. 3): 5
Encrypted message: mtrj

>>>
===== RESTART: D:/T076/CYBER PRAC 1.py =====
=== Classical Cipher Toolkit ===
1. Caesar Cipher - Encrypt
2. Caesar Cipher - Decrypt
3. Monoalphabetic Cipher - Encrypt
4. Monoalphabetic Cipher - Decrypt
Choose 1, 2, 3 or 4: 2
Enter your message: school
Enter shift number (e.g. 3): 3
Decrypted message: pzelli

>>>
===== RESTART: D:/T076/CYBER PRAC 1.py =====
=== Classical Cipher Toolkit ===
1. Caesar Cipher - Encrypt
2. Caesar Cipher - Decrypt
3. Monoalphabetic Cipher - Encrypt
4. Monoalphabetic Cipher - Decrypt
Choose 1, 2, 3 or 4: 3
Enter your message: college
Enter keyword (e.g. SECURITY): boring
Encrypted message: rlhnnan

>>>
===== RESTART: D:/T076/CYBER PRAC 1.py =====
=== Classical Cipher Toolkit ===
1. Caesar Cipher - Encrypt
2. Caesar Cipher - Decrypt
3. Monoalphabetic Cipher - Encrypt
4. Monoalphabetic Cipher - Decrypt
Choose 1, 2, 3 or 4: 4
Enter your message: garden

>>>
===== RESTART: D:/T076/CYBER PRAC 1.py =====
=== Classical Cipher Toolkit ===
1. Caesar Cipher - Encrypt
2. Caesar Cipher - Decrypt
3. Monoalphabetic Cipher - Encrypt
4. Monoalphabetic Cipher - Decrypt
Choose 1, 2, 3 or 4: 3
Enter your message: college
Enter keyword (e.g. SECURITY): boring
Encrypted message: rlhnnan

>>>
===== RESTART: D:/T076/CYBER PRAC 1.py =====
=== Classical Cipher Toolkit ===
1. Caesar Cipher - Encrypt
2. Caesar Cipher - Decrypt
3. Monoalphabetic Cipher - Encrypt
4. Monoalphabetic Cipher - Decrypt
Choose 1, 2, 3 or 4: 4
Enter your message: garden
Enter keyword (e.g. SECURITY): play
Decrypted message: jcsghp

>>>
===== RESTART: D:/T076/CYBER PRAC 1.py =====
=== Classical Cipher Toolkit ===
```

NAME: DEESHA CHAVAN
ROLL NO.: T076

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY

INPUT: CODE FOR GUI BASED METHOD

```
import tkinter as tk
from tkinter import ttk

alphabet = "abcdefghijklmnopqrstuvwxyz"

# Caesar Cipher
def caesar_encrypt(message, shift):
    result = ""
    for letter in message.lower():
        if letter in alphabet:
            old_position = alphabet.index(letter)
            new_position = (old_position + shift) % 26
            result += alphabet[new_position]
        else:
            result += letter
    return result

def caesar_decrypt(message, shift):
    return caesar_encrypt(message, -shift)

# Monoalphabetic (Keyword) Cipher
def build_key_alphabet(keyword):
    keyword = keyword.lower()
    seen = []
    for letter in keyword:
        if letter.isalpha() and letter not in seen:
            seen.append(letter)
    for letter in alphabet:
        if letter not in seen:
            seen.append(letter)
    return "".join(seen)

def mono_encrypt(message, keyword):
    key_alphabet = build_key_alphabet(keyword)
    result = ""
    for letter in message.lower():
        if letter in alphabet:
            position = alphabet.index(letter)
            result += key_alphabet[position]
        else:
            result += letter
```

NAME: DEESHA CHAVAN
ROLL NO.: T076

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY

```
    result += letter  
    return result
```

```
def mono_decrypt(message, keyword):  
    key_alphabet = build_key_alphabet(keyword)  
    result = ""  
    for letter in message.lower():  
        if letter in key_alphabet:  
            position = key_alphabet.index(letter)  
            result += alphabet[position]  
        else:  
            result += letter  
    return result
```

```
# Palette — dark "cipher terminal" theme  
BG = "#0B0F17"  
PANEL = "#131A26"  
PANEL_BORDER = "#22303F"  
NEON_GREEN = "#39FF88"  
NEON_CYAN = "#4CE8FF"  
NEON_AMBER = "#FFC24C"  
TEXT_MAIN = "#D8E1EC"  
TEXT_DIM = "#5F7386"  
DANGER = "#FF5C6C"
```

```
MONO_FONT = "Consolas"
```

```
window = tk.Tk()  
window.title("CIPHER://terminal")  
window.geometry("880x560")  
window.configure(bg=BG)  
window.resizable(False, False)
```

```
style = ttk.Style()  
style.theme_use("clam")  
style.configure("Term.TEntry", font=(MONO_FONT, 11), padding=8,  
                fieldbackground="#0F1520", foreground=NEON_GREEN, insertcolor=NEON_GREEN)  
style.configure("Green.TButton", font=(MONO_FONT, 10, "bold"), foreground=BG,  
                background=NEON_GREEN, borderwidth=0, padding=10)  
style.map("Green.TButton", background=[("active", "#2FE377")])  
style.configure("Cyan.TButton", font=(MONO_FONT, 10, "bold"), foreground=BG,  
                background=NEON_CYAN, borderwidth=0, padding=10)
```

NAME: DEESHA CHAVAN
ROLL NO.: T076

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: CYBER SECURITY

```
style.map("Cyan.TButton", background=[("active", "#33D2F0")])
style.configure("Outline.TButton", font=(MONO_FONT, 10, "bold"), foreground=TEXT_MAIN,
                background="#1A2432", borderwidth=1, padding=10)
style.map("Outline.TButton", background=[("active", "#22303F")])

# Top bar
topbar = tk.Frame(window, bg=BG, height=64)
topbar.pack(fill="x")
topbar.pack_propagate(False)

tk.Label(topbar, text="●", font=(MONO_FONT, 12), fg=DANGER, bg=BG).place(x=20, y=22)
tk.Label(topbar, text="●", font=(MONO_FONT, 12), fg=NEON_AMBER, bg=BG).place(x=38, y=22)
tk.Label(topbar, text="●", font=(MONO_FONT, 12), fg=NEON_GREEN, bg=BG).place(x=56, y=22)

tk.Label(topbar, text="CLASSICAL SUBSTITUTION TECHNIQUES", font=(MONO_FONT, 14, "bold"),
        fg=NEON_GREEN, bg=BG).place(x=90, y=16)
tk.Label(topbar, text="two classical engines · live decrypt/encrypt", font=(MONO_FONT, 8),
        fg=TEXT_DIM, bg=BG).place(x=90, y=40)

# Split panel container
container = tk.Frame(window, bg=BG)
container.pack(fill="both", expand=True, padx=20, pady=(4, 20))

left_panel = tk.Frame(container, bg=PANEL, highlightbackground=PANEL_BORDER,
                      highlightthickness=1)
left_panel.pack(side="left", fill="both", expand=True, padx=(0, 10))

right_panel = tk.Frame(container, bg=PANEL, highlightbackground=PANEL_BORDER,
                      highlightthickness=1)
right_panel.pack(side="left", fill="both", expand=True, padx=(10, 0))

def panel_header(parent, tag, title, color):
    bar = tk.Frame(parent, bg=PANEL)
    bar.pack(fill="x", padx=18, pady=(16, 4))
    tk.Label(bar, text=tag, font=(MONO_FONT, 8, "bold"), fg=BG, bg=color, padx=6,
            pady=2).pack(side="left")
    tk.Label(bar, text=title, font=(MONO_FONT, 12, "bold"), fg=TEXT_MAIN,
            bg=PANEL).pack(side="left", padx=(8, 0))
    sep = tk.Frame(parent, bg=PANEL_BORDER, height=1)
    sep.pack(fill="x", padx=18, pady=(6, 12))

# LEFT: Caesar Cipher
```

NAME: DEESHA CHAVAN

ROLL NO.: T076

```
panel_header(left_panel, " CAESAR ", "Shift Cipher", NEON_GREEN)

tk.Label(left_panel, text="$ message", font=(MONO_FONT, 8, "bold"), fg=TEXT_DIM,
bg=PANEL).pack(anchor="w", padx=18)
caesar_message_entry = ttk.Entry(left_panel, style="Term.TEntry")
caesar_message_entry.pack(fill="x", padx=18, pady=(4, 12))

tk.Label(left_panel, text="$ shift", font=(MONO_FONT, 8, "bold"), fg=TEXT_DIM,
bg=PANEL).pack(anchor="w", padx=18)
caesar_shift_entry = ttk.Entry(left_panel, style="Term.TEntry", width=8)
caesar_shift_entry.insert(0, "3")
caesar_shift_entry.pack(anchor="w", padx=18, pady=(4, 16))

caesar_btn_row = tk.Frame(left_panel, bg=PANEL)
caesar_btn_row.pack(fill="x", padx=18, pady=(0, 14))

caesar_result_var = tk.StringVar(value="awaiting input...")

def caesar_do_encrypt():
    try:
        shift = int(caesar_shift_entry.get())
        caesar_result_var.set(caesar_encrypt(caesar_message_entry.get(), shift))
    except ValueError:
        caesar_result_var.set("ERROR: shift must be an integer")

def caesar_do_decrypt():
    try:
        shift = int(caesar_shift_entry.get())
        caesar_result_var.set(caesar_decrypt(caesar_message_entry.get(), shift))
    except ValueError:
        caesar_result_var.set("ERROR: shift must be an integer")

ttk.Button(caesar_btn_row, text="ENCRYPT »", style="Green.TButton",
command=caesar_do_encrypt).pack(
    side="left", expand=True, fill="x", padx=(0, 6))
ttk.Button(caesar_btn_row, text="« DECRYPT", style="Outline.TButton",
command=caesar_do_decrypt).pack(
    side="left", expand=True, fill="x", padx=(6, 0))

tk.Label(left_panel, text="$ output", font=(MONO_FONT, 8, "bold"), fg=TEXT_DIM,
bg=PANEL).pack(anchor="w", padx=18)
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: CYBER SECURITY

```
caesar_out = tk.Label(left_panel, textvariable=caesar_result_var, font=(MONO_FONT, 11, "bold"),
                      fg=NEON_GREEN, bg="#0F1520", justify="left", anchor="w",
                      wraplength=330, padx=12, pady=12)
caesar_out.pack(fill="x", padx=18, pady=(4, 18))

# RIGHT: Monoalphabetic Cipher
panel_header(right_panel, " MONO ", "Keyword Substitution", NEON_CYAN)

tk.Label(right_panel, text="$ message", font=(MONO_FONT, 8, "bold"), fg=TEXT_DIM,
bg=PANEL).pack(anchor="w", padx=18)
mono_message_entry = ttk.Entry(right_panel, style="Term.TEntry")
mono_message_entry.pack(fill="x", padx=18, pady=(4, 12))

tk.Label(right_panel, text="$ keyword", font=(MONO_FONT, 8, "bold"), fg=TEXT_DIM,
bg=PANEL).pack(anchor="w", padx=18)
mono_keyword_entry = ttk.Entry(right_panel, style="Term.TEntry")
mono_keyword_entry.insert(0, "SECURITY")
mono_keyword_entry.pack(fill="x", padx=18, pady=(4, 8))

mono_mapping_var = tk.StringVar()

def mono_update_mapping(*_):
    kw = mono_keyword_entry.get()
    key_alpha = build_key_alphabet(kw) if kw.strip() else alphabet
    mono_mapping_var.set(f"PLAIN {alphabet.upper()}\nCIPHER {key_alpha.upper()}")

mono_keyword_entry.bind("<KeyRelease>", mono_update_mapping)

tk.Label(right_panel, textvariable=mono_mapping_var, font=(MONO_FONT, 8), fg=NEON_AMBER,
bg="#0F1520", justify="left", anchor="w", padx=12, pady=8).pack(fill="x", padx=18, pady=(0, 14))
mono_update_mapping()

mono_btn_row = tk.Frame(right_panel, bg=PANEL)
mono_btn_row.pack(fill="x", padx=18, pady=(0, 14))

mono_result_var = tk.StringVar(value="awaiting input...")

def mono_do_encrypt():
    keyword = mono_keyword_entry.get()
    if not keyword.strip():
        mono_result_var.set("ERROR: keyword required")
```

NAME: DEESHA CHAVAN

ROLL NO.: T076

```
        return
    mono_result_var.set(mono_encrypt(mono_message_entry.get(), keyword))

def mono_do_decrypt():
    keyword = mono_keyword_entry.get()
    if not keyword.strip():
        mono_result_var.set("ERROR: keyword required")
        return
    mono_result_var.set(mono_decrypt(mono_message_entry.get(), keyword))

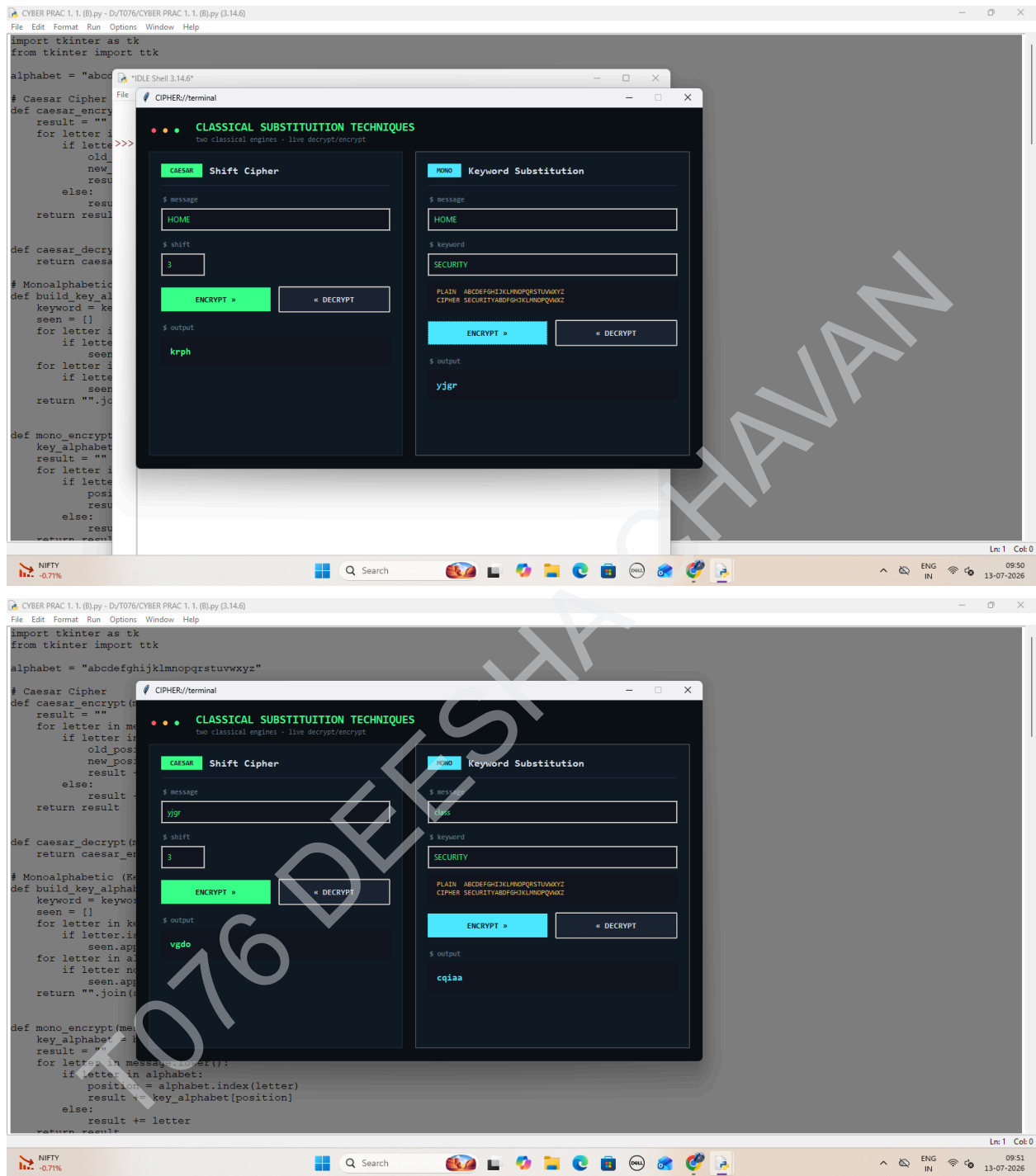
ttk.Button(mono_btn_row, text="ENCRYPT »", style="Cyan.TButton",
command=mono_do_encrypt).pack(
    side="left", expand=True, fill="x", padx=(0, 6))
ttk.Button(mono_btn_row, text="« DECRYPT", style="Outline.TButton",
command=mono_do_decrypt).pack(
    side="left", expand=True, fill="x", padx=(6, 0))

tk.Label(right_panel, text="$ output", font=(MONO_FONT, 8, "bold"), fg=TEXT_DIM,
bg=PANEL).pack(anchor="w", padx=18)
mono_out = tk.Label(right_panel, textvariable=mono_result_var, font=(MONO_FONT, 11, "bold"),
    fg=NEON_CYAN, bg="#0F1520", justify="left", anchor="w",
    wraplength=330, padx=12, pady=12)
mono_out.pack(fill="x", padx=18, pady=(4, 18))

window.mainloop()
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY

OUTPUT:



NAME: DEESHA CHAVAN
ROLL NO.: T076

1. B. Design and implement algorithms to encrypt and decrypt messages using transposition techniques

INPUT: CODE FOR RAIL FENCE AND COLUMNAR TRANSPOSITION TECHNIQUE

import math

Rail Fence Cipher

def rail_encrypt(text, key):

 rail = [['\n' for i in range(len(text))] for j in range(key)]

 row, col = 0, 0

 direction = False

 for ch in text:

 if row == 0 or row == key - 1:

 direction = not direction

 rail[row][col] = ch

 col += 1

 if direction:

 row += 1

 else:

 row -= 1

 result = ""

 for i in range(key):

 for j in range(len(text)):

 if rail[i][j] != '\n':

 result += rail[i][j]

 return result

def rail_decrypt(cipher, key):

 rail = [['\n' for i in range(len(cipher))] for j in range(key)]

 row, col = 0, 0

 direction = None

 for i in range(len(cipher)):

 if row == 0:

 direction = True

 if row == key - 1:

 direction = False

NAME: DEESHA CHAVAN

ROLL NO.: T076

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY

```
rail[row][col] = '*'
col += 1

if direction:
    row += 1
else:
    row -= 1

index = 0
for i in range(key):
    for j in range(len(cipher)):
        if rail[i][j] == '*' and index < len(cipher):
            rail[i][j] = cipher[index]
            index += 1

result = ""
row, col = 0, 0

for i in range(len(cipher)):
    if row == 0:
        direction = True
    if row == key - 1:
        direction = False

    result += rail[row][col]
    col += 1

    if direction:
        row += 1
    else:
        row -= 1

return result

# Columnar Cipher
def columnar_encrypt(message, key):
    col = len(key)
    row = math.ceil(len(message) / col)

    fill = row * col - len(message)
    message += '_' * fill

    matrix = []
    index = 0
```

NAME: DEESHA CHAVAN
ROLL NO.: T076

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY

```
for i in range(row):
    matrix.append(list(message[index:index + col]))
    index += col

order = sorted(list(enumerate(key)), key=lambda x: x[1])

cipher = ""

for index, _ in order:
    for r in range(row):
        cipher += matrix[r][index]

return cipher

def columnar_decrypt(cipher, key):
    col = len(key)
    row = math.ceil(len(cipher) / col)

    matrix = [[" " for i in range(col)] for j in range(row)]

    order = sorted(list(enumerate(key)), key=lambda x: x[1])

    index = 0

    for col_index, _ in order:
        for r in range(row):
            matrix[r][col_index] = cipher[index]
            index += 1

    plain = ""

    for r in range(row):
        for c in range(col):
            plain += matrix[r][c]

    return plain.replace('_', " ")

# Main Program
print("1. Rail Fence Cipher")
print("2. Columnar Transposition Cipher")

choice = int(input("Enter your choice (1/2): "))
```

NAME: DEESHA CHAVAN
ROLL NO.: T076

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY

```
if choice == 1:
    text = input("Enter Plain Text: ").replace(" ", "").upper()
    key = int(input("Enter Number of Rails: "))

    cipher = rail_encrypt(text, key)
    print("Encrypted Text:", cipher)

    plain = rail_decrypt(cipher, key)
    print("Decrypted Text:", plain)

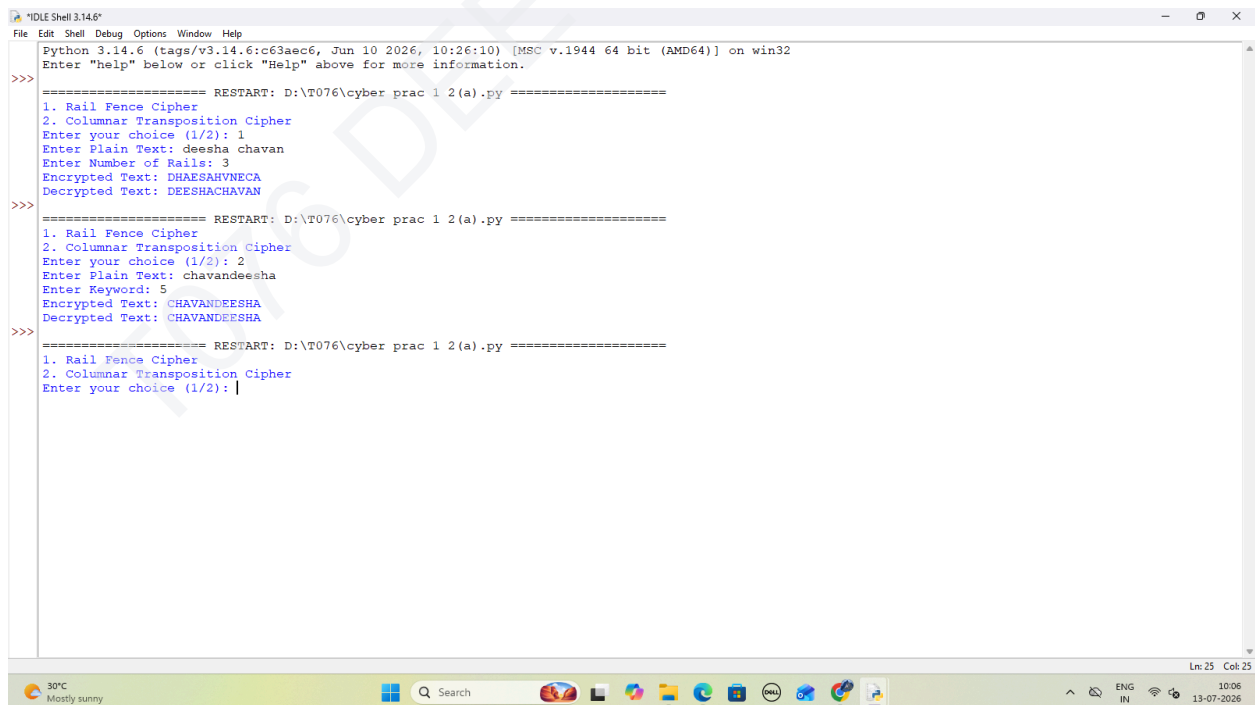
elif choice == 2:
    text = input("Enter Plain Text: ").replace(" ", "").upper()
    key = input("Enter Keyword: ").upper()

    cipher = columnar_encrypt(text, key)
    print("Encrypted Text:", cipher)

    plain = columnar_decrypt(cipher, key)
    print("Decrypted Text:", plain)

else:
    print("Invalid Choice")
```

OUTPUT:



```
"IDLE Shell 3.14.6"
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: D:\T076\cyber prac 1 2(a).py =====
1. Rail Fence Cipher
2. Columnar Transposition Cipher
Enter your choice (1/2): 1
Enter Plain Text: deesha chavan
Enter Number of Rails: 3
Encrypted Text: DHAESHVNECA
Decrypted Text: DEESHACHAVAN
>>> ===== RESTART: D:\T076\cyber prac 1 2(a).py =====
1. Rail Fence Cipher
2. Columnar Transposition Cipher
Enter your choice (1/2): 2
Enter Plain Text: chavandeesh
Enter Keyword: S
Encrypted Text: CHAVANDEESHA
Decrypted Text: CHAVANDEESHA
>>> ===== RESTART: D:\T076\cyber prac 1 2(a).py =====
1. Rail Fence Cipher
2. Columnar Transposition Cipher
Enter your choice (1/2): |
```

NAME: DEESHA CHAVAN
ROLL NO.: T076

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY

INPUT: CODE FOR GUI BASED METHOD

```
import tkinter as tk
from tkinter import ttk, messagebox
import math

# 1. CORE CRYPTOGRAPHY LOGIC
def encrypt_rail_fence(text, key):
    if key <= 1: return text
    rails = [[] for _ in range(len(text))] for _ in range(key)]
    row, step = 0, 1
    for i, char in enumerate(text):
        rails[row][i] = char
        if row == 0: step = 1
        elif row == key - 1: step = -1
        row += step
    return "".join(["".join([c for c in r if c != ""]) for r in rails])

def decrypt_rail_fence(cipher, key):
    if key <= 1: return cipher
    rails = [[] for _ in range(len(cipher))] for _ in range(key)]
    row, step = 0, 1
    for i in range(len(cipher)):
        rails[row][i] = '*'
        if row == 0: step = 1
        elif row == key - 1: step = -1
        row += step
    index = 0
    for r in range(key):
        for c in range(len(cipher)):
            if rails[r][c] == '*' and index < len(cipher):
                rails[r][c] = cipher[index]
                index += 1
    result, row, step = [], 0, 1
    for i in range(len(cipher)):
        result.append(rails[row][i])
        if row == 0: step = 1
        elif row == key - 1: step = -1
        row += step
    return "".join(result)

def encrypt_columnar(text, keyword):
    if not keyword: return text
    num_cols = len(keyword)
```

NAME: DEESHA CHAVAN
ROLL NO.: T076

```

num_rows = math.ceil(len(text) / num_cols)
padded_text = text.ljust(num_rows * num_cols, '_')
grid = [padded_text[i:i+num_cols] for i in range(0, len(padded_text), num_cols)]
col_order = sorted(list(enumerate(keyword)), key=lambda x: x[1])
return ''.join([''.join([row[col_idx] for row in grid]) for col_idx, _ in col_order])

```

```

def decrypt_columnar(cipher, keyword):
    if not keyword: return cipher
    num_cols = len(keyword)
    num_rows = math.ceil(len(cipher) / num_cols)
    grid = [['' for _ in range(num_cols)] for _ in range(num_rows)]
    col_order = sorted(list(enumerate(keyword)), key=lambda x: x[1])
    cipher_idx = 0
    for col_idx, _ in col_order:
        for row_idx in range(num_rows):
            if cipher_idx < len(cipher):
                grid[row_idx][col_idx] = cipher[cipher_idx]
                cipher_idx += 1
    return ''.join([''.join(row) for row in grid]).rstrip('_')

```

2. PRETTY MODERN DARK GUI

```

class ModernCipherApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Transposition Cipher Studio")
        self.root.geometry("650x650")
        self.root.configure(bg="#1e1e2e")

        # Color Palette Definitions
        self.BG_MAIN = "#1e1e2e"
        self.BG_CARD = "#252538"
        self.ACCENT_PURPLE = "#cba6f7"
        self.ACCENT_BLUE = "#89b4fa"
        self.ACCENT_GREEN = "#a6e3a1"
        self.TEXT_MAIN = "#cdd6f4"
        self.TEXT_MUTED = "#a6adc8"
        self.INPUT_BG = "#313244"

        # Global Config for Dropdowns/Inputs
        style = ttk.Style()
        style.theme_use('default')
        style.configure("TCombobox", fieldbackground=self.INPUT_BG, background=self.BG_CARD,
foreground=self.TEXT_MAIN, arrowcolor=self.ACCENT_PURPLE)

```

```
self.setup_ui()

def setup_ui(self):
    # --- TITLE HERO BANNER ---
    title_frame = tk.Frame(self.root, bg=self.BG_MAIN)
    title_frame.pack(fill="x", pady=(25, 15))

    lbl_title = tk.Label(title_frame, text="🔒 TRANSPOSITION STUDIO", font=("Segoe UI", 18, "bold"),
fg=self.ACCENT_PURPLE, bg=self.BG_MAIN)
    lbl_title.pack()
    lbl_subtitle = tk.Label(title_frame, text="Secure processing using Rail Fence & Columnar
techniques", font=("Segoe UI", 10), fg=self.TEXT_MUTED, bg=self.BG_MAIN)
    lbl_subtitle.pack(pady=(2, 0))

    # --- CONTAINER CARD ---
    main_card = tk.Frame(self.root, bg=self.BG_CARD, bd=0, highlightbackground="#313244",
highlightthickness=1)
    main_card.pack(fill="both", expand=True, padx=25, pady=(0, 25))

    # SECTION 1: Algorithm Selector
    lbl_sec1 = tk.Label(main_card, text="ALGORITHM", font=("Segoe UI", 10, "bold"),
fg=self.ACCENT_BLUE, bg=self.BG_CARD)
    lbl_sec1.pack(anchor="w", padx=20, pady=(20, 5))

    self.cipher_choice = tk.StringVar(value="Rail Fence")
    self.dropdown = ttk.Combobox(main_card, textvariable=self.cipher_choice, values=["Rail Fence",
"Columnar Transposition"], state="readonly", font=("Segoe UI", 11))
    self.dropdown.pack(fill="x", padx=20)
    self.dropdown.bind("<<ComboboxSelected>>", self.update_key_helper_text)

    # SECTION 2: Configuration Key
    self.lbl_sec2 = tk.Label(main_card, text="KEY (Number of Rails)", font=("Segoe UI", 10, "bold"),
fg=self.ACCENT_BLUE, bg=self.BG_CARD)
    self.lbl_sec2.pack(anchor="w", padx=20, pady=(15, 5))

    self.entry_key = tk.Entry(main_card, font=("Segoe UI", 11), bg=self.INPUT_BG,
fg=self.TEXT_MAIN, bd=0, insertbackground=self.TEXT_MAIN, highlightthickness=1,
highlightbackground="#45475a")
    self.entry_key.pack(fill="x", padx=20, ipady=6)
    self.entry_key.insert(0, "3")

    # SECTION 3: Input Field
    lbl_sec3 = tk.Label(main_card, text="INPUT STRING", font=("Segoe UI", 10, "bold"),
fg=self.ACCENT_BLUE, bg=self.BG_CARD)
```

```
lbl_sec3.pack(anchor="w", padx=20, pady=(15, 5))

self.txt_input = tk.Text(main_card, height=4, font=("Segoe UI", 11), bg=self.INPUT_BG,
fg=self.TEXT_MAIN, bd=0, insertbackground=self.TEXT_MAIN, highlightthickness=1,
highlightbackground="#45475a", wrap="word")
self.txt_input.pack(fill="x", padx=20)

# ACTION BUTTONS GRID
btn_frame = tk.Frame(main_card, bg=self.BG_CARD)
btn_frame.pack(fill="x", padx=20, pady=20)

btn_encrypt = tk.Button(btn_frame, text="EXECUTE ENCRYPTION", font=("Segoe UI", 10, "bold"),
bg=self.ACCENT_PURPLE, fg=self.BG_MAIN, activebackground="#b4befe",
activeforeground=self.BG_MAIN, bd=0, cursor="hand2", command=self.run_encrypt)
btn_encrypt.pack(side="left", expand=True, fill="x", padx=(0, 5), ipady=8)

btn_decrypt = tk.Button(btn_frame, text="EXECUTE DECRYPTION", font=("Segoe UI", 10, "bold"),
bg=self.ACCENT_GREEN, fg=self.BG_MAIN, activebackground="#94e2d5",
activeforeground=self.BG_MAIN, bd=0, cursor="hand2", command=self.run_decrypt)
btn_decrypt.pack(side="right", expand=True, fill="x", padx=(5, 0), ipady=8)

# SECTION 4: Output Field
lbl_sec4 = tk.Label(main_card, text="OUTPUT RESULT", font=("Segoe UI", 10, "bold"),
fg=self.ACCENT_BLUE, bg=self.BG_CARD)
lbl_sec4.pack(anchor="w", padx=20, pady=(5, 5))

self.txt_output = tk.Text(main_card, height=4, font=("Segoe UI", 11, "bold"), bg="#181825",
fg=self.ACCENT_PURPLE, bd=0, highlightthickness=1, highlightbackground="#313244",
wrap="word")
self.txt_output.pack(fill="x", padx=20, pady=(0, 20))

def update_key_helper_text(self, event=None):
    if self.cipher_choice.get() == "Rail Fence":
        self.lbl_sec2.config(text="KEY (Number of Rails - e.g., 3)")
        self.entry_key.delete(0, tk.END)
        self.entry_key.insert(0, "3")
    else:
        self.lbl_sec2.config(text="KEY (Alphabetic Word Phase - e.g., SECRET)")
        self.entry_key.delete(0, tk.END)
        self.entry_key.insert(0, "SECRET")

def validate_inputs(self):
    text = self.txt_input.get("1.0", tk.END).strip()
    key_raw = self.entry_key.get().strip()
```

```
if not text:
    messagebox.showwarning("Empty Input", "Please provide a message inside the Input text
area.")
    return None, None

if self.cipher_choice.get() == "Rail Fence":
    try:
        key = int(key_raw)
        if key <= 1: raise ValueError
        return text, key
    except ValueError:
        messagebox.showerror("Key Type Mis-match", "Rail Fence algorithm demands a positive
whole integer greater than 1.")
        return None, None
    else:
        if not key_raw.isalpha():
            messagebox.showerror("Key Type Mis-match", "Columnar Transposition requires a purely
text alphabet keyword string (A-Z).")
            return None, None
        return text, key_raw

def run_encrypt(self):
    text, key = self.validate_inputs()
    if text is None: return

    if self.cipher_choice.get() == "Rail Fence":
        res = encrypt_rail_fence(text, key)
    else:
        res = encrypt_columnar(text, key)

    self.txt_output.config(fg=self.ACCENT_PURPLE)
    self.txt_output.delete("1.0", tk.END)
    self.txt_output.insert(tk.END, res)

def run_decrypt(self):
    text, key = self.validate_inputs()
    if text is None: return

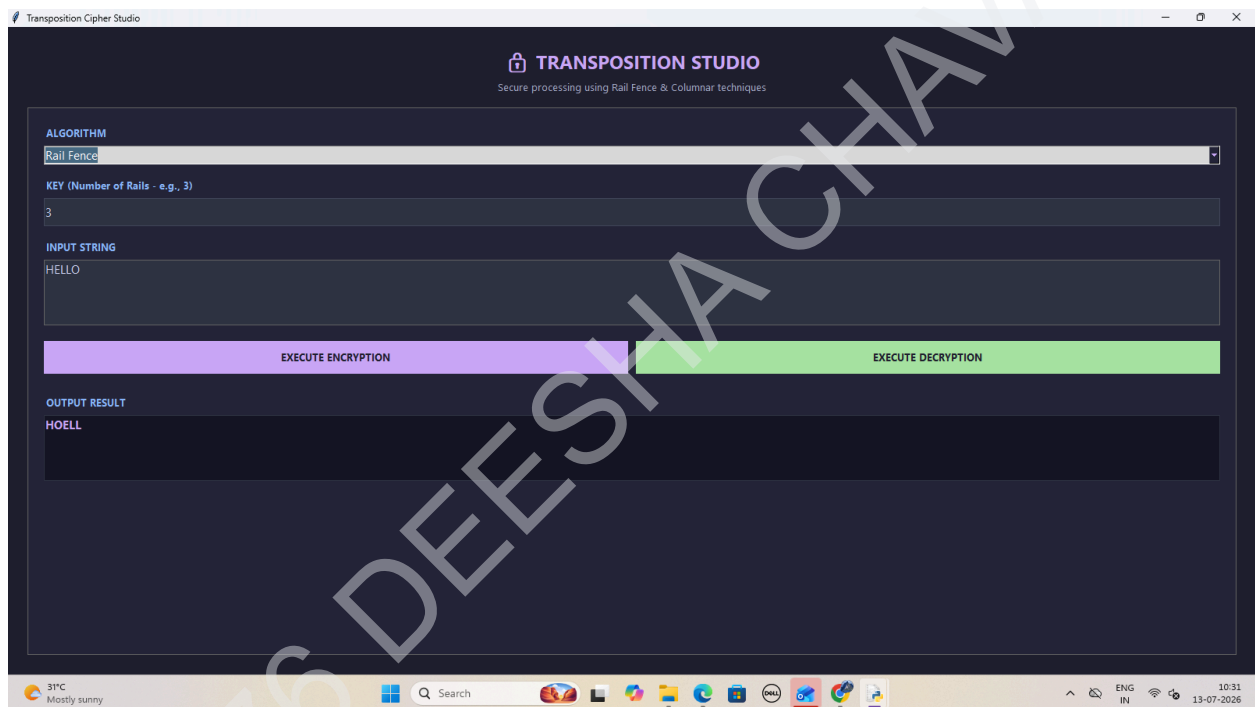
    if self.cipher_choice.get() == "Rail Fence":
        res = decrypt_rail_fence(text, key)
    else:
        res = decrypt_columnar(text, key)
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY

```
self.txt_output.config(fg=self.ACCENT_GREEN)  
self.txt_output.delete("1.0", tk.END)  
self.txt_output.insert(tk.END, res)
```

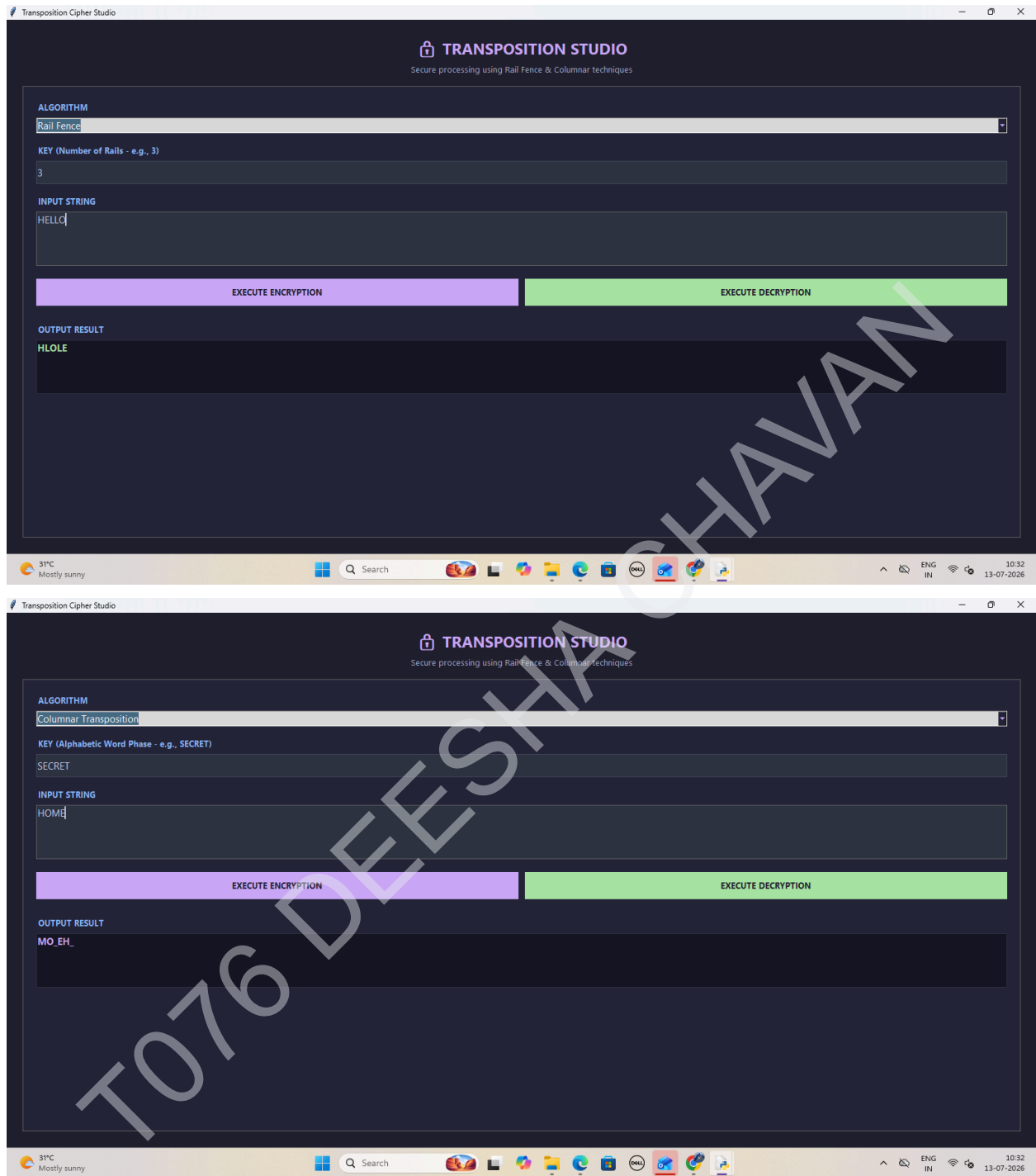
```
if __name__ == "__main__":  
    root = tk.Tk()  
    app = ModernCipherApp(root)  
    root.mainloop()
```

OUTPUT:



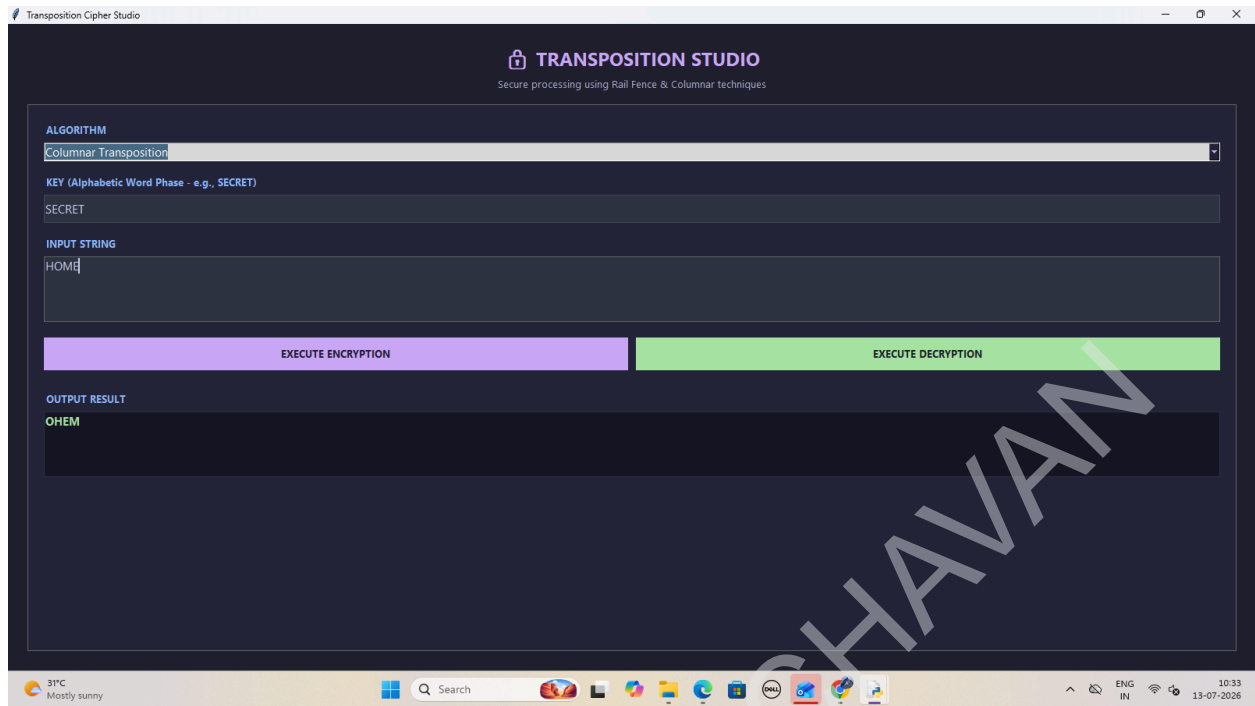
NAME: DEESHA CHAVAN
ROLL NO.: T076

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY



NAME: DEESHA CHAVAN
ROLL NO.: T076

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT: CYBER SECURITY



NAME: DEESHA CHAVAN
ROLL NO.: T076